

正则化

正则化简单来说就是：原本模型只追求训练 Loss 小，但我们再给复杂模型加一个代价，让模型倾向于学习更简单、更稳定、更容易泛化的规律，防止过拟合的情况。

一、过拟合（Overfitting）

过拟合就是模型把训练数据学得太死了，连训练数据里的偶然噪声和细枝末节都记住了，结果遇到没见过的新数据反而表现不好，缺少泛化能力。换句话说，就是只会做这套题，题目稍微一变就不会做了。

发生过拟合的常见原因一般有两个：模型拥有大量参数、表现力强，以及训练数据少。

二、权值衰减

我们可以用权值衰减的手段来抑制过拟合，这种方法就是对学习过程中较大的参数进行惩罚，从而抑制过拟合，因为过拟合很多时候都是由于权重参数过大导致的。

大概思路就是，给最后输出的 Loss 值加上一个惩罚，比如加上一个权重的平方范数（L2 范数）具体为：将权重记为 W ，L2 范数的权值衰减就是：

$$L_{\text{reg}} = L + \frac{1}{2} \lambda \|W\|_2^2$$

然后将这个 $\frac{1}{2} \lambda \|W\|_2^2$ 加到损失函数上。其中， λ 是控制正则化强度的超参数，其大小决定着权重惩罚的程度，而 $\frac{1}{2}$ 则是为了将 $\frac{1}{2} \lambda \|W\|_2^2$ 的求导值调整为 λW 。

L2 范数相当于各个元素的平方和的平方根。用数学式表示的话，假设有权重：

$$W = (w_1, w_2, \dots, w_n)$$

则 L2 范数为：

$$\|W\|_2 = \sqrt{w_1^2 + w_2^2 + \dots + w_n^2} = \sqrt{\sum_{i=1}^n w_i^2}$$

权值衰减中使用的是 L2 范数的平方：

$$\|W\|_2^2 = \sum_{i=1}^n w_i^2$$

除了 L2 范数，还有 L1 范数、 L_∞ 范数等。L1 范数是各个元素的绝对值之和，相当于：

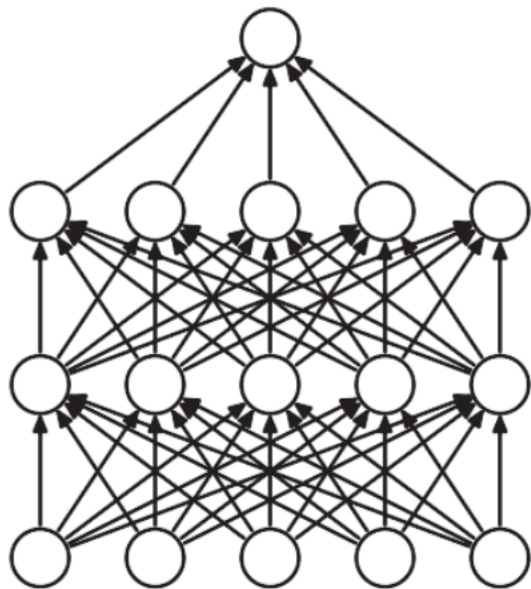
$$\|W\|_1 = |w_1| + |w_2| + \dots + |w_n|$$

L_∞ 范数也称为 Max 范数，相当于各个元素的绝对值中最大的那一个：

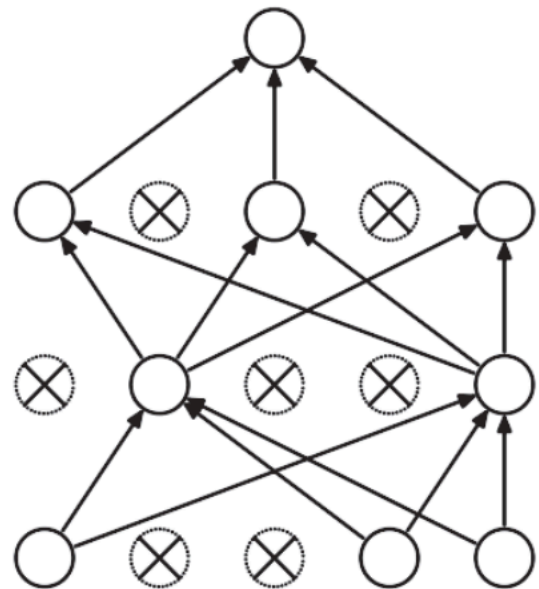
$$\|W\|_\infty = \max_i |w_i|$$

三、Dropout

Dropout 是一种在学习的过程中随机删除神经元的方法。训练时，每一次随机选出隐藏层的神经元，然后将其删除。如图，被删除的神经元依旧存在，只是这一次训练中不再传递信号。



(a) Standard Neural Net



(b) After applying dropout.

我们用 Python 代码简单实现一下这个功能：

```
class Dropout:
    def __init__(self, dropout_ratio=0.5):
        self.dropout_ratio = dropout_ratio
        self.mask = None

    def forward(self, x, train_flg=True):
        # train_flg 表示是否正在训练，训练时随机删除神经元，但预测时不可以
        if train_flg:
            self.mask = np.random.rand(*x.shape) > self.dropout_ratio
            return x * self.mask
        else:
            # 由于训练时输出是损失一部分后的结果，因此预测时也应和训练时保持一致
            return x * (1.0 - self.dropout_ratio)

    def backward(self, dout):
        # 传回未被删除的神经元的梯度
        return dout * self.mask
```

Dropout 与集成学习

机器学习中经常使用集成学习，即让多个模型单独进行学习，推理时再取多个模型的输出的平均值。经实验验证，通过进行集成学习，神经网络的识别精度可以提高很多。我

们可以将 Dropout 理解为，通过在学习过程中随机删除神经元，从而实现每一次都让不同的模型进行学习。也就是说，Dropout 通过一个网络模拟实现了集成学习的效果。